

Smart Factory Machinery Predictor

Scenario: You are analyzing sensor data from a manufacturing plant's assembly line. The machinery has various settings (categorical) and outputs continuous sensor readings (temperature, vibration, RPM). The plant manager wants a dual-purpose tool: one that predicts the internal temperature of a machine to prevent overheating, and another that classifies whether a manufactured part will pass or fail quality inspection based on those same machine conditions.

The Problem Statement: "Using an industrial sensor dataset, build a machine learning pipeline using `scikit-learn` that predicts a continuous metric (e.g., machine temperature) using multivariable regression. Additionally, build a logistic regression classifier to predict a binary outcome (e.g., product failure). You must handle categorical machine settings, properly split your data, research the impact of gradient descent learning rates using standard library tools, and save the final models for future deployment."

Task 1: Simple Linear Regression

- **The Goal:** Establish the baseline of predictive modeling.
- **The Action:** Start with a clean subset of the factory data. Train a simple linear regression model to predict the machine's internal **Temperature** using just a single continuous variable, such as **RPM**.
- **The Research:** Have the student plot the line of best fit and calculate the Mean Squared Error (MSE) to establish a baseline performance.

Task 2: Multivariable Regression

- **The Goal:** Introduce complexity and multi-dimensional data.
- **The Action:** Upgrade the model to predict Temperature using multiple continuous variables simultaneously (e.g., RPM, Vibration, and Pressure).
- **The Research:** Compare the MSE of this new model against the simple model from Task 1. The student must document why adding more relevant features usually (but not always) reduces the error rate.

Smart Factory Machinery Predictor

Task 3: Save Model

- **The Goal:** Bridge the gap between a local script and a deployable asset.
- **The Action:** Take the best-performing multivariable model from the morning session and serialize it to the hard drive using libraries like [pickle](#) or [joblib](#).
- **The Research:** Write a short script in a separate file to load this saved model and predict the temperature for a mock "new" machine reading. This proves the model is detached from the training environment.

Task 4: One-Hot Encoding

- **The Goal:** Learn to handle real-world, non-numeric factory data.
- **The Action:** Introduce a categorical column to the dataset (e.g., "Machine Mode: Eco, Standard, High-Speed"). Use encoding techniques to convert these text labels into binary columns (\$1\$s and \$0\$s) so a machine learning algorithm can process them.
- **The Research:** Explain the "Dummy Variable Trap" and why one column is often dropped after encoding.

Task 5: Train-Test Split

- **The Goal:** Prevent the model from merely memorizing the data (overfitting).
- **The Action:** Take the newly encoded, complex dataset and strictly divide it (e.g., 80% for training, 20% for testing) before feeding it to any algorithm.
- **The Research:** Train a new multivariable model on the 80% split and test it on the 20%. The student must document why evaluating a model on the exact same data it was trained on is a critical failure in data science.

Task 6: Logistic Regression

- **The Goal:** Shift from predicting a continuous number to predicting a distinct category.
- **The Action:** Using the properly split and encoded dataset, train a Logistic Regression classifier to predict a binary outcome: **1** if a manufactured part will be **Defective**, and **0** if it will be **Good**.
- **The Research:** Generate a Confusion Matrix for the test set predictions. The student must analyze the real-world factory cost of a False Positive (throwing away a good part) versus a False Negative (shipping a broken part to a customer).